



Universidad Nacional Experimental de Guayana

Coordinación General de Pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

Sección: 1

Tema 2 - Los Lenguajes de Programación

Profesor:

Felix Marquez

Alumnos:

Iván Hernández - C.I: 30.529.821

Miguel Gómez - C.I: 29.862.177

Miguel Barrios - C.I: 28.619.898

Daniel Valenzuela - C.I: 28.619.852

Puerto Ordaz, 26 de Mayo de 2026.

Resumen Académico

La investigación inicia analizando el paradigma imperativo y estructurado como el reflejo directo de la arquitectura de Von Neumann, caracterizado por una ejecución lineal de instrucciones vinculada al ciclo de la CPU y la manipulación destructiva de la memoria física mediante el Stack y el Heap, lo que genera riesgos latentes como la explosión de estados y efectos secundarios incontrolables derivados del aliasing. En contraposición a la fragilidad procedimental, se examina cómo el paradigma orientado a objetos introduce la alta cohesión y el bajo acoplamiento al unificar datos y algoritmos en entidades autónomas encargadas de proteger sus propias invariantes de estado, detallando su motor interno de despacho dinámico mediante tablas de métodos virtuales y argumentando los beneficios de la composición flexible por inyección de dependencias frente a la rigidez taxonómica de la herencia tradicional.

Avanzando hacia modelos con fundamentos puramente matemáticos, el documento profundiza en el paradigma funcional, inspirado en el cálculo lambda de Alonzo Church, el cual erradica las variables mutables a favor de valores inmutables gestionados con compartición estructural, promoviendo la transparencia referencial, las funciones de orden superior y la evaluación perezosa a través de thunks que permiten el procesamiento seguro de colecciones de datos infinitas.

Finalmente, el informe contrasta la teoría de la ciencia de la computación con la práctica de la ingeniería de software a través de un riguroso benchmarking basado en la Conjetura de Collatz ejecutado sobre un hardware modesto, lo que permite validar empíricamente que la jerarquía de rendimiento favorece la compilación nativa de Rust y Zig sobre las optimizaciones en tiempo de ejecución (JIT) de JavaScript en el motor V8 y el esquema interpretado de Python, demostrando el impacto directo que el layout de memoria, los costes de abstracción y la gestión de recursos poseen sobre la eficiencia actual.

Introducción

En el ámbito de la ingeniería en informática, los lenguajes de programación constituyen las herramientas fundamentales para modelar soluciones y abstraer la complejidad del hardware subyacente. Sin embargo, más allá de la sintaxis particular de cada lenguaje, el aspecto definitorio de la arquitectura del software radica en los paradigmas de programación, entendidos como los marcos conceptuales y doctrinas de diseño que guían la forma en que se estructuran el pensamiento lógico, los datos y el flujo de control computacional. El estudio sistemático de estos paradigmas permite comprender no solo cómo se ejecutan los programas a bajo nivel, sino también bajo qué garantías operan en términos de seguridad, predecibilidad y mantenibilidad.

El presente informe ofrece un análisis exhaustivo y profundo de los paradigmas fundamentales que configuran las ciencias de la computación modernas. La investigación abarca desde el modelo clásico Imperativo y Estructurado, íntimamente ligado a la arquitectura clásica de von Neumann y la manipulación lineal y mutable del estado de la memoria , hasta el enfoque Orientado a Objetos (POO), concebido para mitigar la fragilidad del código procedural mediante la unificación de datos y comportamientos bajo fronteras de alta cohesión y bajo acoplamiento. Asimismo, se explora el rigor matemático del paradigma Funcional, fundamentado en el cálculo lambda y la inmutabilidad de los datos , y el paradigma Lógico/Declarativo, sustentado en el cálculo de predicados y la resolución formal automatizada. Adicionalmente, ante las exigencias de procesamiento de las arquitecturas multinúcleo contemporáneas, se examina el Modelo de Actores como una alternativa robusta para la computación concurrente y distribuida basada en el aislamiento absoluto de estados.

Finalmente, con el propósito de contrastar la teoría computacional con la praxis de la ingeniería de software, se incorpora un riguroso análisis sintáctico y empírico. A través de

especificaciones en Notación de Backus-Naur (BNF) y un estudio comparativo de rendimiento (*benchmarking*) basado en la Conjetura de Collatz, se evalúan los lenguajes Zig, Rust, Python y JavaScript, evidenciando el impacto directo que poseen las estrategias de compilación, interpretación y gestión de memoria sobre la eficiencia del software actual.

Matriz descriptiva y análisis paradigmático

Paradigma Imperativo/Estructural: El Control del Estado y la Máquina

El paradigma imperativo es el reflejo más directo de la arquitectura clásica de Von Neumann. Se fundamenta en la premisa de que un programa es una secuencia de instrucciones secuenciales que modifican el estado de la máquina para alcanzar un objetivo computacional.

Ejecución Paso a Paso y el Ciclo de Instrucción

En este modelo, el flujo de control es explícito y lineal, interrumpido únicamente por estructuras de control como ciclos y bifurcaciones. La CPU opera mediante el ciclo de fetch-decode-execute (búsqueda, decodificación y ejecución):

1. Fetch: La Unidad de Control lee la instrucción apuntada por el *Program Counter* (PC) desde la memoria principal.
2. Decode: Se descomponen los opcodes (códigos de operación) y los operandos.
3. Execute: La Unidad Aritmético-Lógica (ALU) ejecuta la operación, impactando los registros o la memoria, y el PC se incrementa para apuntar a la dirección física de la siguiente instrucción.

El programador dictamina exactamente cómo se debe resolver el problema, guiando a la máquina a través de una transición de estados finitos donde cada instrucción matemática o de movimiento de datos representa un vector de cambio en el tiempo.

Del Imperativismo Puro a la Programación Estructurada

En los albores de la computación, el flujo se alteraba mediante saltos incondicionales (GOTO). Esto reflejaba fielmente el código de máquina, pero generaba el denominado *Spaghetti Code*, un entramado inmanejable de transferencias de control.

La evolución hacia el subparadigma estructurado se consolidó gracias al Teorema de Böhm-Jacopini, el cual demuestra matemáticamente que cualquier función computable puede ser implementada utilizando únicamente tres estructuras de control:

- Secuencia: Ejecución sucesiva de instrucciones (A seguido de B).
- Selección: Bifurcación condicional (if-then-else) basada en el valor lógico de una expresión.
- Iteración: Repetición de un bloque de instrucciones (while, for) mientras se cumpla un predicado.

Este avance conceptual eliminó la necesidad del GOTO, permitiendo que el código imperativo se pudiera leer e interpretar de arriba hacia abajo, alineando la estructura del texto del programa con su flujo de ejecución en tiempo real.

Mutabilidad y Modificación Directa de la Memoria

La esencia del imperativismo radica en la asignación destructiva. Cuando se ejecuta una sentencia como $x = x + 1$, no se está estableciendo una ecuación o igualdad matemática, sino una orden explícita de mutación de hardware.

A nivel de bajo nivel, este proceso involucra una interacción directa con los registros del procesador y los buses de datos:

El estado global del programa se define matemáticamente como un mapeo continuo entre identificadores (variables) y ubicaciones de memoria física. Al ser destructiva, la

información previa se pierde irremediablemente, lo que obliga al desarrollador a gestionar el ciclo de vida de cada variable con extrema precisión.

El Impacto del Layout de Memoria: Stack vs. Heap

La mutabilidad imperativa se manifiesta de forma distinta dependiendo de la región de memoria donde se alojan los datos, un factor crítico para el rendimiento y la estabilidad del sistema:

Región de Memoria	Mecanismo de Control	Tipo de Datos	Características y Riesgos
Stack (Pila)	Asignación automática por el compilador mediante el <i>Stack Pointer</i> .	Variables locales y marcos de llamadas a funciones.	Ultra veloz, de tamaño fijo. Riesgo latente de <i>Stack Overflow</i> si se abusa de la recursión profunda.
Heap (Montículo)	Asignación dinámica en tiempo de ejecución (ej. <code>malloc()</code> , <code>new</code>).	Estructuras de datos complejas o de tamaño variable.	Flexible pero lenta. Requiere liberación manual o recolector de basura (<i>Garbage Collector</i>). Riesgo de fragmentación y <i>Memory Leaks</i> .

La manipulación directa de estas regiones mediante punteros (variables que almacenan direcciones físicas de memoria) otorga al paradigma estructural un poder absoluto sobre el hardware, permitiendo técnicas como la aritmética de punteros, pero abriendo la puerta a errores críticos de seguridad como desbordamientos de búfer (*Buffer Overflows*) o punteros colgantes (*Dangling Pointers*).

Efectos Secundarios (Side Effects) y Explosión de Estados

Un efecto secundario ocurre cuando la invocación de una subrutina o la evaluación de una expresión altera un componente del estado que se encuentra más allá de su entorno local inmediato.

- **Impacto:** Si una función modifica una variable global, escribe en un canal de Entrada/Salida (I/O), altera un archivo en disco o modifica un objeto pasado por referencia, está generando un efecto secundario.
- **Consecuencias:** Aunque optimiza el uso de recursos al reutilizar celdas de memoria en lugar de clonar datos, destruye la capacidad de analizar el código de forma aislada. El comportamiento de una función deja de ser predecible; ya no depende solo de sus parámetros de entrada, sino del momento exacto del tiempo (el orden de ejecución de los hilos o sentencias previas) en que es invocada.

El Problema del Aliasing

El *aliasing* es un fenómeno imperativo crítico que multiplica la peligrosidad de los efectos secundarios. Ocurre cuando una misma ubicación de memoria es accesible a través de dos o más identificadores distintos (por ejemplo, dos punteros apuntando a la misma dirección).

Si la función `modificar_datos(puntero_A)` altera el valor apuntado, el programador que observa el comportamiento de `puntero_B` verá que su valor ha cambiado de forma aparentemente mágica y sin previo aviso, rompiendo el principio de razonamiento local.

Visualización del Cambio de Estado: Tabla de Traza

Para auditar la mutabilidad y los efectos secundarios, el paradigma imperativo depende de las tablas de traza, las cuales registran cronológicamente el valor de las variables tras la ejecución de cada línea de código.

Tomemos como ejemplo el siguiente algoritmo estructural que calcula un factorial interactuando con variables globales:

La mutación del estado interno del programa a lo largo del tiempo de ejecución se puede modelar formalmente en la siguiente tabla:

Paso de Ejecución	Línea de Código	Variable n (Input)	Variable Local i	Variable Global <code>global_result</code>
0	<i>Estado Inicial</i>	3	<i>Indefinido</i>	1
1	<code>int i = 1;</code>	3	1	1
2	<code>while (i <= 3)</code> <i>(Evaluación: True)</i>	3	1	1
3	<code>global_result = 1</code> <code>* 1;</code>	3	1	1

4	i = 1 + 1;	3	2	1
5	while (2 <= 3) (Evaluación: True)	3	2	1
6	global_result = 1 * 2;	3	2	2
7	i = 2 + 1;	3	3	2
8	while (3 <= 3) (Evaluación: True)	3	3	2
9	global_result = 2 * 3;	3	3	6
10	i = 3 + 1;	3	4	6
11	while (4 <= 3) (Evaluación: False)	3	4	6

Esta traza lineal evidencia la explosión de estados: un programa con N variables mutables de tipo booleano puede existir teóricamente en 2^N estados diferentes. A medida que el número de variables y subrutinas con efectos secundarios crece, verificar la corrección

del software se vuelve una tarea exponencialmente compleja para el cerebro humano, lo que impulsó el nacimiento de paradigmas alternativos como el funcional o el declarativo.

Paradigma Orientado a Objetos (POO): Cohesión, Abstracción y Flexibilidad

El paradigma orientado a objetos surge para mitigar la dispersión y fragilidad del código imperativo/estructural, unificando los datos y los algoritmos que los manipulan en entidades conceptuales coherentes llamadas objetos. Su adopción resuelve la desconexión existente en el software procedural, transformando sistemas complejos en redes de componentes autónomos que colaboran entre sí.

La Unión de Datos y Comportamiento: Cohesión y Acoplamiento

En el desarrollo procedural clásico, las estructuras de datos (structs o registros) son componentes pasivos que viajan a través de una red de funciones activas. Esto genera dos problemas críticos: la dispersión lógica y el alto acoplamiento. La POO invierte esta dinámica mediante el principio de cohesión conceptual: los datos y la lógica que los altera deben residir en el mismo lugar.

- **Alta Cohesión:** Al agrupar atributos y métodos relacionados bajo una misma frontera (la clase), se garantiza que cada componente del sistema tenga una única responsabilidad claramente definida.
- **Bajo Acoplamiento:** Al interactuar exclusivamente a través de interfaces, los objetos limitan el conocimiento que tienen sobre el funcionamiento interno de los demás. Si la estructura interna de un objeto cambia, el resto del sistema permanece inalterado, reduciendo el "efecto dominó" durante el mantenimiento.

El Concepto Original: Paso de Mensajes

Aunque la sintaxis moderna utiliza la invocación de métodos (`objeto.metodo()`), la concepción original de la POO (formulada por Alan Kay en Smalltalk) visualizaba a los objetos como células biológicas o computadoras independientes en una red. Los objetos no "llaman a funciones"; se envían mensajes de forma asíncrona. Es el objeto receptor el que decide, de manera autónoma en tiempo de ejecución, cómo responder a dicho mensaje.

Encapsulamiento Profundo y Protección de Invariantes

El encapsulamiento no debe reducirse al uso cosmético de modificadores de acceso como `private`, `protected` o `public`. Su verdadero propósito en la ingeniería de software es el ocultamiento de la información (*Information Hiding*) y la preservación de invariantes.

¿Qué es una Invariante de Estado?

Una invariante es una condición o regla de negocio que debe permanecer estrictamente verdadera durante todo el ciclo de vida de un objeto (excepto temporalmente mientras se ejecuta un método interno).

Ejemplo Crítico: En una clase `CuentaBancaria`, una invariante podría dictar que el balance jamás puede ser menor que el `limiteDeSobregiro`.

Si el atributo `balance` fuera público, cualquier agente externo podría ejecutar `cuenta.balance = -99999`, rompiendo la invariante y corrompiendo la integridad del modelo. Al privatizar el atributo y forzar la interacción a través de un método como `retirarEfectivo(monto)`, el objeto actúa como un guardián de su propio estado, validando la operación antes de aplicar la mutación.

El Antipatrón de los Getters y Setters Indiscriminados

Un error común de diseño consiste en generar automáticamente métodos *getter* y *setter* para cada atributo privado. Esto destruye el encapsulamiento profundo, transformando la clase en una estructura anémica (una estructura de datos pasiva disfrazada de objeto). El diseño robusto de objetos sigue el principio "Tell, Don't Ask" (Dile, no le preguntues): no le pidas sus datos a un objeto para tomar una decisión externa; dile al objeto lo que debe hacer basándose en su propio estado.

Polimorfismo y el Mecanismo de Despacho Dinámico

El polimorfismo de subtipado o dinámico permite que una abstracción genérica adopte múltiples formas concretas. Permite escribir código que interactúe con una superclase o interfaz abstracta, delegando el comportamiento específico a las clases derivadas.

El Motor Interno: La Tabla de Métodos Virtuales (VTABLE)

Para entender cómo el sistema sabe qué método ejecutar sin que esté escrito explícitamente en el código binario, los lenguajes compilados basados en POO (como C++ o Java) implementan la VTABLE.

Cuando una clase define un método virtual (un método susceptible de ser sobrescrito), el compilador realiza los siguientes pasos de bajo nivel:

1. Creación de la VTABLE: Construye una tabla estática para esa clase que contiene un array de punteros de memoria apuntando a las implementaciones concretas de sus métodos.

2. Inyección del VPTR: Añade un puntero oculto llamado vptr (Virtual Pointer) dentro de la estructura física de cada objeto instanciado. Este vptr apunta directamente a la VTABLE de su clase correspondiente.

Cuando se ejecuta el código, el proceso de Despacho Dinámico (*Dynamic Dispatch*) resuelve la dirección en tiempo de ejecución:

El compilador traduce la llamada `p->presentarse()` en una resolución indirecta similar a: `*(p->vptr[index_presentarse])(p)`. Esto garantiza el desacoplamiento absoluto: el emisor solo necesita conocer el índice del método en la interfaz, mientras que el hardware resuelve dinámicamente la dirección física del código ejecutable.

Discusión Clásica: Herencia vs. Composición

La gestión de la reutilización de código divide a la POO en dos vertientes arquitectónicas: la extensión taxonómica (Herencia) y la agregación funcional (Composición).

El Problema de la Clase Base Frágil (Fragile Base Class)

La herencia representa una relación rígida de acoplamiento estructural conocida como "Caja Blanca", dado que las clases hijas suelen depender de los detalles de implementación interna de la superclase.

Si un equipo de desarrollo modifica la optimización de un método en la clase base, esa pequeña alteración puede propagarse destructivamente hacia abajo, rompiendo el comportamiento de decenas de clases derivadas que dependían de la secuencia de ejecución

original. Esto viola el principio de encapsulamiento, ya que la subclase está expuesta a las tripas de su ancestro.

Violación del Principio de Sustitución de Liskov (LSP)

El abuso de la herencia suele quebrar el rigor matemático del diseño. El ejemplo clásico es la jerarquía del Cuadrado y el Rectángulo:

- Geométricamente, un Cuadrado es *un* Rectángulo.
- Si se modela class Cuadrado extends Rectangulo, la clase Rectangulo expondrá los métodos setAncho(w) y setAlto(h).
- Un objeto Cuadrado no puede implementar dichos métodos de forma independiente sin violar su propia naturaleza (alterar el ancho implica alterar el alto).

Si un algoritmo recibe un objeto de tipo Rectangulo y modifica su ancho esperando que el alto permanezca constante, y en su lugar recibe un Cuadrado, el programa fallará catastróficamente. La relación taxonómica del mundo real ha fallado en el software.

La Flexibilidad Dinámica de la Composición

La composición implementa un acoplamiento de "Caja Negra". Los objetos se comunican exclusivamente a través de sus interfaces públicas sin conocer sus estructuras internas.

- Inyección de Dependencias: Al diseñar mediante composición, un objeto puede recibir sus componentes en el constructor. Esto permite cambiar el comportamiento de un sistema completo en tiempo de ejecución simplemente inyectando una implementación de interfaz distinta (por ejemplo, cambiar un componente MotorGasolina por un

MotorElectrico), algo matemáticamente imposible de lograr con la rigidez estática de la herencia.

Dimensión Analítica	Herencia (is-a)	Composición (has-a)
Tiempo de Definición	Estática (Tiempo de Compilación).	Dinámica (Tiempo de Ejecución).
Visibilidad	Caja Blanca (Rompe encapsulamiento interno).	Caja Negra (Respeta fronteras públicas).
Grado de Acoplamiento	Extremadamente Fuerte / Rígido.	Débil / Altamente Flexible.
Estrategia de Extensión	Modificar la jerarquía de clases.	Injectar nuevos componentes e interfaces.

Paradigma Funcional: La Computación como Evaluación Matemática

Inspirado en el cálculo lambda (λ -cálculo) de Alonzo Church, el paradigma funcional concibe la programación no como una serie de modificaciones consecutivas a la memoria, sino como la evaluación de funciones matemáticas puras, libres de estado mutable. Mientras que el paradigma imperativo se alinea con la Máquina de Turing (estados y transiciones de cinta), el funcional se basa en sistemas formales de sustitución y ligadura de variables.

Inmutabilidad de los Datos y Compartición Estructural

En el paradigma funcional puro, las variables en el sentido imperativo (celdas de memoria mutables) no existen; existen los valores (datos atómicos o compuestos) y las uniones (*bindings*), que son asignaciones de nombres a valores de carácter permanente y de solo lectura.

El Mecanismo de Compartición Estructural (*Structural Sharing*)

La regla estricta de la inmutabilidad dicta que si se requiere modificar una estructura de datos (como una lista o un árbol), es imposible alterar la instancia original. En su lugar, se debe generar una nueva estructura que refleje el cambio. Para evitar que esto resulte en un cuello de botella con una complejidad temporal y espacial de $O(N)$ debido a la clonación indiscriminada, los lenguajes funcionales implementan Estructuras de Datos Persistentes.

Estas estructuras (comúnmente árboles de tipo *Hash Array Mapped Tries* o HAMTs) utilizan la compartición estructural:

- Al insertar, eliminar o modificar un elemento, el compilador o entorno de ejecución clona únicamente el nodo afectado y el camino de nodos que lleva hasta la raíz (un proceso conocido como *path copying*).
- Los nodos que no sufrieron alteraciones no se duplican; la nueva versión de la estructura simplemente apunta a las subestructuras de la versión anterior.

Gracias a este mecanismo, las operaciones de "modificación" en colecciones inmutables se ejecutan con una eficiencia de $O(\log N)$ o incluso casi constante $O(1)$, garantizando la seguridad frente a la concurrencia sin penalizar drásticamente el rendimiento del hardware.

Funciones como Ciudadanos de Primer Orden y de Orden Superior

Que las funciones sean ciudadanos de primer orden (*First-Class Functions*) significa que reciben el mismo trato computacional que tipos primitivos como los enteros o las cadenas de caracteres.

Composición y Parametrización Funcional

Esta propiedad habilita la existencia de las funciones de orden superior (*Higher-Order Functions*), las cuales se definen como funciones que reciben otras funciones como argumentos, devuelven funciones como resultado, o ambas cosas. Las tres operaciones fundamentales de este ecosistema son:

- Map: Aplica una transformación $f: X \rightarrow Y$ a cada elemento de una colección de tipo X, devolviendo una nueva colección de tipo Y.
- Filter: Evalúa cada elemento de una colección mediante un predicado booleano $p: X \rightarrow B$, reteniendo solo aquellos elementos que devuelven verdadero.
- Reduce / Fold: Acumula los elementos de una estructura combinándolos secuencialmente mediante una función binaria $g: (A, X) \rightarrow A$, reduciendo la colección a un único valor atómico.

Currificación (*Currying*) y Aplicación Parcial

Derivado directamente de la lógica del λ -cálculo, donde todas las funciones formalmente reciben un solo argumento, surge la currificación. Es el proceso de transformar una función que acepta múltiples argumentos en una secuencia de funciones que aceptan un único argumento cada una.

Si se tiene una función matemática $f(a, b, c)$, su versión curried se evalúa como $f(a)(b)(c)$. Esto facilita la aplicación parcial, que consiste en fijar los primeros argumentos de una función para crear una nueva función más especializada, promoviendo la reutilización de lógica algorítmica sin necesidad de declarar subrutinas adicionales.

Transparencia Referencial y Razonamiento Ecuacional

Una expresión posee transparencia referencial si puede ser sustituida directamente por su valor resultante en cualquier punto del programa sin alterar en absoluto el comportamiento del sistema. Esto exige de forma mandatoria que las funciones involucradas sean puras:

1. **Determinismo Absoluto:** Para un conjunto dado de argumentos de entrada, la función devolverá exactamente el mismo resultado, sin importar cuántas veces o en qué momento cronológico sea invocada.
2. **Ausencia de Efectos Secundarios:** La ejecución no altera el estado de la memoria, no modifica variables externas, ni realiza operaciones ocultas de Entrada/Salida (I/O).

Razonamiento Ecuacional (*Equational Reasoning*)

La transparencia referencial permite aplicar el razonamiento ecuacional. El programador y el compilador pueden tratar el código como si fuera álgebra elemental. Si sabemos que $f(x) = 2x$, la expresión $f(5) + f(5)$ puede optimizarse de forma segura como $10 + 10$ o como $2 \times f(5)$, sabiendo con certeza matemática que el entorno no alterará su comportamiento.

Característica	Enfoque Imperativo (Función Impura)	Enfoque Funcional (Función Pura)

Dependencia Temporal	Alta. El resultado depende del estado de la memoria al invocarla.	Nula. Depende exclusivamente de los parámetros de entrada.
Optimización por Compilador	Limitada por el riesgo de romper efectos secundarios ocultos.	Avanzada. Permite <i>Dead Code Elimination</i> y <i>Loop Fusion</i> .
Memoización	Inviabile sin un control complejo de invalidación de caché.	Nativa y segura. Los resultados pueden cachearse indefinidamente.
Pruebas Unitarias	Requieren entornos simulados (<i>Mocks</i>) y control de estado.	Triviales. Solo requieren verificar entradas contra salidas.

Evaluación Perezosa (*Lazy Evaluation*) frente a Evaluación Estricta

Los lenguajes de programación difieren en su estrategia de evaluación de expresiones. Mientras que la mayoría utiliza la evaluación estricta (*Eager Evaluation*), donde los argumentos de una función se calculan por completo antes de ingresar al cuerpo de la misma, lenguajes funcionales puros (como Haskell) implementan la evaluación perezosa o por necesidad.

Mecánica Interna: Los *Thunks*

Cuando el compilador se encuentra con una expresión bajo un esquema perezoso, no genera instrucciones de cálculo inmediato. En su lugar, envuelve la expresión en una estructura de datos interna denominada Thunk (o suspensión). El *thunk* contiene:

- El puntero al código de la expresión.
- Las ligaduras de las variables necesarias para su cómputo.

El cálculo efectivo se pospone hasta el momento exacto en que otra sección del programa requiere el valor (por ejemplo, para imprimirlo en pantalla o usarlo en una condición lógica). Una vez evaluado el *thunk*, este se sobrescribe destructivamente con el valor resultante (un proceso de memoización automática a nivel de runtime) para asegurar que no se compute dos veces.

Estructuras de Datos Infinitas (*Streams*)

La consecuencia arquitectónica más potente de la evaluación perezosa es la capacidad de modelar colecciones infinitas de datos de forma segura. Considérese la generación de la sucesión de Fibonacci de manera infinita:

Haskell

Bajo evaluación estricta, esta línea provocaría un desbordamiento de memoria instantáneo al intentar llenar el *heap*. Bajo evaluación perezosa, la lista existe como una receta de generación latente. Si el programa ejecuta `take 5 fibs`, el motor de ejecución evaluará los *thunks* estrictamente necesarios para extraer los primeros cinco elementos (`[0, 1, 1, 2, 3]`), dejando el resto de la lista en estado latente e inofensivo en memoria.

Paradigma Lógico/Declarativo: Programación Basada en Relaciones

A diferencia de los enfoques basados en la computación imperativa u orientada a objetos, el paradigma lógico se sitúa plenamente en el espectro declarativo. Su fundamentación matemática no reside en la Máquina de Turing ni en el λ -cálculo, sino en el Cálculo de Predicados de Primer Orden. En este modelo, el programador exime a la máquina de instrucciones algorítmicas explícitas; en su lugar, describe formalmente las verdades del dominio del problema mediante relaciones, delegando la deducción de las soluciones a un motor de inferencia automatizado.

Relaciones, Hechos y la Base de Conocimientos

Un programa lógico no se compone de funciones o procedimientos, sino de una Base de Conocimientos (KB, *Knowledge Base*) estructurada a partir de axiomas lógicos. Los elementos sintácticos de este modelo (característicos de lenguajes como Prolog) se clasifican rigurosamente en:

- Términos Atómicos (Átomos y Números): Constantes unívocas que representan objetos del mundo real (ej. juan, maria, 5).
- Variables: Identificadores que comienzan con mayúscula (ej. X, Y) que actúan como marcadores de posición susceptibles de ser ligados a un valor.
- Términos Compuestos (Estructuras): Un functor seguido de argumentos (ej. fecha(3, junio, 2026)).

Hechos frente a Reglas

La base de conocimientos articula estos términos mediante dos estructuras lógicas:

1. Hechos: Afirmaciones absolutas e incondicionales sobre el dominio. Establecen relaciones n-arias que se asumen axiomáticamente verdaderas (ej. padre(juan, maria)).
2. Reglas: Sentencias condicionales que dictaminan cómo inferir nuevas verdades a partir de los hechos existentes. Tienen la forma conceptual de *"Para que la conclusión sea verdadera, el cuerpo de condiciones debe cumplirse"*.

Prolog

La Hipótesis del Mundo Cerrado (Closed World Assumption)

El entorno de ejecución lógico opera bajo la Hipótesis del Mundo Cerrado (CWA). Esto significa que la base de conocimientos se considera un universo completo y absoluto de la verdad. Si el sistema intenta demostrar una consulta y no encuentra una ruta lógica de hechos o reglas que la respalde, la consulta no se categoriza como "desconocida", sino como falsa. Esta propiedad da origen a la *Negación como Fallo* (NAF): algo es falso simplemente porque el sistema falló en demostrar su veracidad.

El Motor de Inferencia: Unificación y Cláusulas de Horn

Para transformar las declaraciones lógicas en computación activa, el paradigma depende de un motor de inferencia que procesa las consultas de forma matemática a través de dos mecanismos: la unificación y la resolución.

El Rigor de las Cláusulas de Horn

Las reglas de un programa lógico se restringen formalmente a las Cláusulas de Horn. En lógica de predicados, una cláusula de Horn es una disyunción de literales donde a lo sumo

uno de los literales es positivo (no negado). Matemáticamente, una regla condicional se expresa originalmente como:

Aplicando las leyes de la lógica booleana, esta equivalencia se transforma en una implicación directa:

En la sintaxis de la programación lógica, esta implicación se invierte por legibilidad (donde A es la "Cabeza" y los literales B componen el "Cuerpo"): $A :- B_1, B_2, \dots, B_n$. Esta restricción matemática es crítica, ya que simplifica drásticamente la complejidad computacional del proceso de demostración automática.

El Algoritmo de Unificación

La unificación es un procedimiento de emparejamiento sintáctico bidireccional entre dos términos. El motor intenta encontrar una sustitución de variables (un conjunto de ligaduras denotado como θ) que haga que ambas expresiones sean idénticas.

- Si se intenta unificar $\text{padre}(\text{juan}, X)$ con $\text{padre}(\text{juan}, \text{maria})$, el algoritmo tiene éxito aplicando la sustitución $\theta = \{X \rightarrow \text{maria}\}$.
- Si se intenta unificar con $\text{padre}(\text{pedro}, \text{maria})$, el algoritmo falla inmediatamente por conflicto de constantes.

El problema del *Occurs Check*: Un aspecto crítico de la unificación es determinar si una variable X se encuentra contenida dentro del propio término con el que se intenta unificar (por ejemplo, intentar unificar X con $f(X)$). Teóricamente, esto requeriría una estructura infinita ($X = f(f(f(\dots)))$). Por motivos de optimización de rendimiento, muchos motores de ejecución omiten por defecto el *occurs check*, lo que puede provocar bucles infinitos o comportamientos impredecibles a menos que se invoque explícitamente.

Resolución SLD y Árboles de Búsqueda

La demostración de una consulta se ejecuta mediante Resolución SLD (*Selected Linear Definite clause resolution*). El motor toma la meta propuesta por el usuario e intenta realizar un encadenamiento hacia atrás (*backward chaining*):

1. Busca en la KB una cabeza de regla o hecho que unifique con la meta.
2. Si unifica con una regla, la meta original es reemplazada por el cuerpo de dicha regla, generando una lista de submetas.
3. El proceso se repite de forma recursiva utilizando una estrategia de Búsqueda en Profundidad (DFS) sobre el árbol de soluciones.

Cuando una ruta de unificación conduce a un callejón sin salida (un fallo), el motor activa el Backtracking. El sistema retrocede cronológicamente al último "punto de elección" (*Choice Point*), deshace las ligaduras de variables realizadas en esa rama fallida y explora la siguiente alternativa disponible en la base de conocimientos.

El Control de Flujo Implícito y la Ecuación de Kowalski

El cambio de paradigma más radical de la programación lógica se resume en la célebre ecuación dictada por el científico de la computación Robert Kowalski:

Algoritmo = Lógica + Control

La Lógica: Representa las relaciones, hechos y reglas del problema. Es responsabilidad absoluta del programador.

- El Control: Representa la estrategia de ejecución, el orden de búsqueda y el manejo de la memoria. Es responsabilidad absoluta del sistema/runtime.

Criterio de Evaluación	Enfoque Imperativo / Estructural	Enfoque Lógico / Declarativo
Mecanismo de Control	Explícito (Bucles for, while, condicionales, saltos).	Implícito (Resolución SLD, Búsqueda DFS automática).
Flujo de Datos	Unidireccional (Los parámetros de entrada producen salidas).	Multidireccional / Bidireccional (Las variables pueden actuar como entradas o salidas dinámicamente).
Estructura de Repetición	Iteración de bajo nivel o llamadas a funciones recursivas.	Recursión pura integrada con <i>Backtracking</i> nativo.
Estado de Ejecución	Mutación continua de celdas físicas de memoria.	Sustitución y ligadura progresiva de variables lógicas libres.

La Flexibilidad de los Predicados Multidireccionales

Debido a la ausencia de un flujo de control restrictivo, los programas lógicos poseen la propiedad única de ser multidireccionales. Un mismo predicado puede ser reutilizado para múltiples propósitos computacionales dependiendo de qué variables se pasen ligadas o libres al momento de hacer la consulta.

Considérese el predicado $\text{sumar}(X, Y, Z)$, gobernado por la relación matemática $X + Y = Z$. El sistema puede resolver escenarios radicalmente distintos sin cambiar una sola línea de código:

Prolog

Desviaciones Pragmáticas: El Operador de Corte (*Cut*)

A pesar de la belleza matemática de la lógica pura, los sistemas reales se ejecutan en hardware con recursos finitos. Para optimizar el espacio en memoria (evitando la acumulación excesiva de puntos de elección en el *stack*), lenguajes como Prolog introducen operadores de control manual directos, rompiendo parcialmente el purismo declarativo:

- El Operador Corte (!): Fuerza al motor de inferencia a podar el árbol de búsqueda de forma destructiva. Al pasar por un corte, el sistema se compromete con las elecciones tomadas hasta ese momento, prohibiendo cualquier intento posterior de *backtracking* sobre las ramas previas.
- Cortes Verdes frente a Cortes Rojos: Un "corte verde" es aquel que se añade únicamente para mejorar la eficiencia temporal del compilador sin alterar las soluciones lógicas del programa. Un "corte rojo" altera drásticamente el significado semántico del código; si se remueve, el programa no solo se vuelve más lento, sino que empieza a devolver respuestas lógicamente incorrectas, reintroduciendo sutilmente el pensamiento imperativo dentro de un entorno declarativo.

Paradigma Concurrente/Basado en Actores: Aislamiento y Tolerancia a Fallos

La computación moderna, impulsada por las arquitecturas multinúcleo y los sistemas distribuidos a gran escala, requiere manejar múltiples flujos de ejecución simultáneos. Mientras que la concurrencia tradicional basada en hilos (*threads*) imperativos sufre por los dolores de

cabeza del estado compartido mutable (bloqueos, semáforos, *deadlocks*), el Modelo de Actores —propuesto originalmente por Carl Hewitt en 1973 y popularizado por lenguajes como Erlang (Joe Armstrong) y frameworks como Akka— propone una alternativa radical asentada sobre el aislamiento total y la comunicación matemática.

Anatomía de un Actor y el Ciclo de Procesamiento Secuencial

En este modelo, la unidad fundamental de computación y primitivo de concurrencia ya no es una línea secuencial de código que comparte memoria, sino el actor. Un actor es una entidad computacional autónoma que encapsula estrictamente tres elementos esenciales:

- Estado (*State*): Los datos internos que definen la condición del actor en un momento dado.
- Comportamiento (*Behavior*): La lógica algorítmica que determina cómo reaccionará el actor cuando reciba un mensaje específico.
- Cola de Mensajes (*Mailbox*): Un búfer o cola con estructura FIFO (*First-In, First-Out*) donde se almacenan los mensajes entrantes dirigidos a ese actor.

Restricción Estricta de No Compartición

Un actor no puede, bajo ninguna circunstancia, acceder, leer o modificar el estado interno de otro actor. No existen variables globales compartidas ni punteros cruzados. Cada proceso concurrente funciona como una isla independiente con su propio hilo de ejecución lógico. El estado mutable deja de ser global y se vuelve estrictamente local, eliminando de raíz la necesidad de sincronizaciones forzadas a nivel de hardware.

Las Tres Acciones Fundamentales (Leyes de Hewitt)

Cuando un actor procesa un mensaje de su *mailbox*, la ejecución de su comportamiento puede realizar única y exclusivamente una combinación finita de las siguientes tres acciones básicas:

1. **Enviar Mensajes:** Enviar un número finito de mensajes asíncronos a otros actores cuyas direcciones (*ActorRefs*) conozca de antemano.
2. **Crear Actores (*Spawn*):** Instanciar un número finito de nuevos actores hijos, heredando o recibiendo dinámicamente capacidades operativas.
3. **Designar el Siguiete Comportamiento:** Especificar la estrategia lógica o modificaciones de estado que se aplicarán para procesar el *siguiete* mensaje de la cola. Esto permite mutaciones conceptuales de estado de forma inmutable (por ejemplo, mutar mediante la sustitución del comportamiento en una máquina de estados).

Elixir

Paso de Mensajes Asíncronos frente a Condiciones de Carrera

Toda comunicación e interacción dentro de un sistema de actores se realiza de forma exclusiva mediante el paso de mensajes asíncronos e inmutables.

Mecánica del Envío No Bloqueante

Cuando el actor A requiere que el actor B realice una tarea o le provea información:

- **Emisión Libre:** A envía un mensaje a la dirección de B y continúa de inmediato con su propia ejecución. El emisor jamás se bloquea esperando que el receptor procese la orden.

- Encolamiento Atómico: El mensaje viaja por el entorno de ejecución y se deposita de forma segura en el *mailbox* de B.
- Procesamiento Secuencial: El actor B extrae los mensajes de su *mailbox* de uno en uno. Aunque N actores envíen mensajes concurrentemente a B, la ejecución interna de B es estrictamente unihilo respecto a su cola.

Erradicación de Condiciones de Carrera (Race Conditions)

Las condiciones de carrera ocurren en entornos imperativos cuando dos o más hilos intentan modificar simultáneamente una celda de memoria, rompiendo la previsibilidad del estado final debido al entrelazamiento de instrucciones a bajo nivel.

Al eliminar la memoria compartida mediante el aislamiento absoluto y canalizar toda interacción a través de una cola secuencial atómica por actor, las condiciones de carrera quedan erradicadas por diseño. No se requieren mecanismos complejos de sincronización manual (mutex, variables de condición, cerrojos), lo que permite que el sistema escale de manera masiva y lineal sin degradar el rendimiento por contención de bloqueos.

Tolerancia a Fallos y la Filosofía "Let It Crash"

La concurrencia tradicional maneja los errores mediante estructuras try-catch incrustadas en los mismos hilos de ejecución. Si un hilo sufre una excepción no controlada, muere de forma destructiva, dejando a menudo los recursos compartidos en un estado corrupto o permanentemente bloqueados.

El modelo de actores propone un paradigma radicalmente distinto basado en la resiliencia sistémica: "Let It Crash" (Deja que falle).

El Principio de Supervisión Jerárquica

Dado que un fallo en un actor está completamente contenido dentro de sus propias fronteras de aislamiento, su muerte no contamina el estado del resto del sistema. En lugar de intentar prevenir defensivamente cada posible error, los actores se organizan en estructuras de árbol denominadas Árboles de Supervisión (*Supervision Trees*).

Un actor supervisor es responsable del ciclo de vida de sus actores hijos. Si un hijo sufre una anomalía y colapsa:

1. El hijo muere limpiamente, liberando sus recursos locales.
2. Envía automáticamente una señal de fallo (*exit signal*) a su supervisor.
3. El supervisor, operando en un flujo aislado, intercepta la señal y ejecuta una estrategia de recuperación predefinida.

Estrategias de Supervisión Estándar

- One-for-One (Uno por Uno): Si un actor hijo falla, solo ese actor específico es reiniciado (*restarted*) desde un estado limpio conocido. Los actores hermanos continúan operando sin interrupción.
- One-for-All (Uno por Todos): Si un hijo falla, el supervisor mata a todos los demás actores hermanos coordinados bajo esa rama y los reinicia a todos en cascada. Se utiliza cuando los actores de ese nivel poseen una alta dependencia operativa entre sí.
- Rest-for-One (El Resto por Uno): Si un hijo falla, se reinician únicamente los actores que fueron creados cronológicamente *después* del actor afectado, respetando el orden lineal de las dependencias.

Transparencia de Ubicación (Location Transparency)

Una de las mayores fortalezas arquitectónicas del modelo de actores es la transparencia de ubicación. El identificador o dirección de un actor (*ActorRef*) es una abstracción lógica pura que oculta la ubicación física real de la entidad.

Para el actor emisor, la sintaxis y semántica para enviar un mensaje es idéntica si el receptor se está ejecutando en:

- El mismo núcleo del procesador.
- Un hilo diferente dentro de la misma Máquina Virtual.
- Un servidor de bases de datos remoto al otro lado del planeta.

El *runtime* del sistema de actores se encarga de serializar el mensaje y enviarlo a través de sockets TCP/IP de forma transparente si detecta que la dirección pertenece a un nodo distribuido. Esto permite diseñar sistemas de software que pueden escalar verticalmente (añadiendo más núcleos de CPU) u horizontalmente (añadiendo más servidores al clúster) de manera transparente, sin necesidad de refactorizar la lógica de negocio fundamental del programa.

Análisis Comparativo de Concurrencia

Dimensión de Análisis	Concurrencia por Memoria Compartida (Hilos)	Concurrencia Basada en Actores
Primitiva Base	Hilos físicos/lógicos del Sistema Operativo (Thread, pthreads).	Actores (Procesos ligeros administrados por un <i>runtime</i>).

Gestión del Estado	Global o Compartido. Mutable en memoria física común.	Aislado por completo. Mutable solo de forma local/inmutable.
Mecanismo de Sincronización	Manual e Imperativo (Locks, Semaphores, Mutex).	Implícito y Automatizado mediante colas FIFO (<i>Mailboxes</i>).
Riesgos Latentes	Condición de carrera, Interbloqueo (<i>Deadlock</i>), Inversión de prioridad.	Acumulación masiva de mensajes pendientes en el <i>Mailbox</i> (OOM).
Filosofía ante Errores	Prevención defensiva local mediante bloques try-catch.	Tolerancia a fallos reactiva mediante Árboles de Supervisión (<i>Let it crash</i>).
Capacidad de Distribución	Limitada estrictamente a una sola máquina física.	Nativa y Transparente a nivel de red (<i>Location Transparency</i>).

Código fuente analizado de Zig, Rust, Python y Javascript junto a la tabla de Benchmarking

Para el benchmarking se seleccionó la Conjetura de Collatz sobre todo $n < 100\,000$. El algoritmo es computacionalmente intensivo con bifurcación condicional densa y aritmética entera, lo que lo hace ideal para medir diferencias reales de rendimiento entre mecanismos de ejecución. Entorno de hardware (pruebas Python, JavaScript, Zig y Rust — ejecución real): Intel Celeron 5205U (2 núcleos), 4 GB RAM, Windows 10 22H2, Python 3.12.3, Node.js 20 (motor V8 JIT), Zig 0.13.0 y Rust 1.96.0.

Zig

- **Palabras reservadas (selección):** const, var, fn, pub, return, if, else, while, for, switch, break, continue, defer, errdefer, try, catch, comptime, inline, extern, packed, struct, enum, union, error, null, undefined, true, false, unreachable, anytype, test, noalias, threadlocal.
- **Reglas de identificadores:** Patrón `[a-zA-Z_][a-zA-Z0-9_]*`. Sensible a mayúsculas/minúsculas. No se permite Unicode sin escape `@"..."`. Los identificadores que colisionen con palabras reservadas deben encomillarse: `@"var"`.
- **Literales soportados:** Enteros decimales, hexadecimales (0x...), octales (0o...), binarios (0b...) con separador `_` (100_000). Flotantes con notación científica. Cadenas con comillas dobles. Caracteres con comillas simples. Booleanos true/false. null y undefined para tipos opcionales y no inicializados.
- **Delimitadores de bloque:** Llaves `{}` para todos los bloques. Punto y coma `;` para terminar sentencias. Sin indentación significativa. La ausencia de un recolector de

basura obliga al programador a usar defer para liberar recursos automáticamente al salir del scope.

Gramática BNF simplificada — estructuras esenciales en Zig:

```
FunctionDecl ::= ("pub")? "fn" IDENT "(" ParamList? ")" ReturnType Block

ReturnType  ::= "void" | "!" Type | Type

IfStmt     ::= "if" "(" Expr ")" Block ("else" (IfStmt | Block))?

WhileStmt   ::= "while" "(" Expr ")" (":" "(" Stmt+ ")" )? Block

ForStmt     ::= "for" "(" Expr ")" "!" IDENT "!" Block

Block      ::= "{" Stmt* "}"

Stmt       ::= VarDecl | Assign | IfStmt | WhileStmt | ReturnStmt | CallExpr ";"

VarDecl    ::= ("const" | "var") IDENT (":" Type)? "=" Expr ";"
```

Código fuente Zig — Conjetura de Collatz:

```
const std = @import("std");

fn collatzPasos(n_inicial: u64) u64 {
    var n = n_inicial; var pasos: u64 = 0;

    while (n != 1) {
        if (n % 2 == 0) { n /= 2; } else { n = 3 * n + 1; }

        pasos += 1;
    }

    return pasos;
}

pub fn main() !void {
    const stdout = std.io.getStdOut().writer();

    const N: u64 = 100_000;
```

```

var timer = try std.time.Timer.start();

var max_pasos: u64 = 0; var numero_max: u64 = 1; var i: u64 = 1;

while (i < N) : (i += 1) {

    const pasos = collatzPasos(i);

    if (pasos > max_pasos) { max_pasos = pasos; numero_max = i; }

}

const transcurrido_ns = timer.read();

const tiempo_ms = @as(f64, @floatFromInt(transcurrido_ns)) / 1_000_000.0;

try stdout.print("Conjetura de Collatz verificada para todo 1 <= n < {} \n", .{N});

try stdout.print("Numero con mas pasos: {} ({} pasos) \n", .{ numero_max, max_pasos });

try stdout.print("Tiempo de ejecucion: {:.2} ms \n", .{tiempo_ms});

}

```

Python

- **Palabras reservadas (35 en Python 3.12):** False, None, True, and, as, assert, async, await, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield.
- **Reglas de identificadores:** Patrón `[a-zA-Z_][a-zA-Z0-9_]*`. Soporta identificadores Unicode (PEP 3131). Los identificadores con doble guion bajo (`__dunder__`) tienen semántica especial del intérprete. Convención: `snake_case` para variables/funciones, `PascalCase` para clases.
- **Literales soportados:** int (precisión arbitraria), float (IEEE 754 doble precisión), complex (j sufijo), str (comillas simples/dobles/triples), bytes (prefijo b), bool (True/False), None. Separador numérico: `1_000_000`.
- **Delimitadores y bloques:** Python utiliza indentación significativa como delimitador de bloques (INDENT/DEDENT tokens generados por el lexer). Los bloques se abren con dos puntos `:` y se cierran por dedentación. Esto hace al análisis léxico dependiente del

contexto de indentación, una propiedad inusual entre los lenguajes de propósito general. No se usan llaves ni palabras clave de cierre.

- **Diferencia crítica con los LLM:** La tokenización en Python para el compilador (CPython) opera sobre una gramática formal LR(1) donde INDENT/DEDENT son tokens explícitos generados por el analizador léxico. En contraste, los LLM tokenizan usando BPE (Byte Pair Encoding) sobre texto plano, sin semántica gramatical.

Gramática BNF simplificada — estructuras esenciales en Python:

```
FunctionDecl ::= "def" IDENT "(" ParamList? ")" ("->" Type)? ":" Suite
IfStmt      ::= "if" Expr ":" Suite
              ("elif" Expr ":" Suite)*
              ("else" ":" Suite)?
WhileStmt   ::= "while" Expr ":" Suite ("else" ":" Suite)?
ForStmt     ::= "for" TargetList "in" Expr ":" Suite ("else" ":" Suite)?
Suite       ::= NEWLINE INDENT Stmt+ DEDENT
Stmt        ::= SimpleStmt | CompoundStmt
```

Código fuente Python — Conjetura de Collatz:

```
import time

def collatz_pasos(n: int) -> int:
    pasos = 0
    while n != 1:
        if n % 2 == 0: n //= 2
        else:      n = 3 * n + 1
        pasos += 1
```

```

        return pasos

inicio = time.perf_counter()

max_p, num_max = 0, 1

for i in range(1, 100_000):

    p = collatz_pasos(i)

    if p > max_p: max_p, num_max = p, i

print(f"Tiempo: {(time.perf_counter()-inicio)*1000:.2f} ms")

```

Rust

- **Palabras reservadas activas:** as, break, const, continue, crate, else, enum, extern, false, fn, for, if, impl, in, let, loop, match, mod, move, mut, pub, ref, return, self, Self, static, struct, super, trait, true, type, unsafe, use, where, while, async, await, dyn.
- **Palabras reservadas para uso futuro:** abstract, become, box, do, final, macro, override, priv, try, typeof, unsized, virtual, yield. Reservarlas sin implementar demuestra la planificación cuidadosa de la evolución del lenguaje.
- **Reglas de identificadores:** Patrón `[a-zA-Z_][a-zA-Z0-9_]*`. Soporta Unicode NFKC normalizado. Convención: snake_case para variables/funciones, PascalCase para tipos/traits. El compilador emite advertencias por convenciones violadas.
- **Literales soportados:** Enteros con sufijos de tipo explícito (42u8, 1000i32, 0xFFu64). Flotantes con sufijos f32/f64. Cadenas &str y String (UTF-8 garantizado). Caracteres char (Unicode scalar value, comillas simples). Bytes b'x41'. Booleanos true/false.
- **Delimitadores:** Llaves {} para bloques de código. El punto y coma ; distingue sentencias de expresiones: la última expresión sin ; en un bloque es el valor de retorno implícito. Este detalle sintáctico es semánticamente crucial y afecta directamente al análisis del parser.

Gramática BNF simplificada — estructuras esenciales en Rust:

```
FunctionDecl ::= ("pub")? "fn" IDENT Generics? "(" ParamList? ")" ("->") Type)? Block

IfExpr      ::= "if" Expr Block ("else" (IfExpr | Block))?

WhileExpr    ::= "while" Expr Block

LoopExpr     ::= "loop" Block

ForExpr      ::= "for" Pattern "in" Expr Block

MatchExpr    ::= "match" Expr "{" MatchArm+ "}"

MatchArm     ::= Pattern ("if" Expr)? "=>" (Expr ", " | Block)

Block        ::= "{" Stmt* Expr? "}"
```

Código fuente Rust — Conjetura de Collatz:

```
use std::time::Instant;

fn collatz_pasos(mut n: u64) -> u64 {
    let mut pasos: u64 = 0;

    while n != 1 {
        if n % 2 == 0 { n /= 2; } else { n = 3 * n + 1; }

        pasos += 1;
    }

    pasos
}

fn main() {
    let inicio = Instant::now();

    let (mut max_p, mut num_max) = (0u64, 1u64);

    for i in 1..100_000u64 {
        let p = collatz_pasos(i);

        if p > max_p { max_p = p; num_max = i; }
    }
}
```

```
    }  
    println!("Tiempo: {:.2} ms", inicio.elapsed().as_secs_f64() * 1000.0);  
}
```

JavaScript

- **Palabras reservadas (ECMAScript 2024):** break, case, catch, class, const, continue, debugger, default, delete, do, else, enum, export, extends, false, finally, for, function, if, import, in, instanceof, let, new, null, return, static, super, switch, this, throw, true, try, typeof, var, void, while, with, yield, async, await, of, implements, interface, package, private, protected, public.
- **Reglas de identificadores:** Patrón `[$_a-zA-Z][\w$]*`. Soporta Unicode completo (cualquier letra definida en Unicode). Los identificadores pueden comenzar con `$` o `_` (convenciones de jQuery y variables privadas respectivamente). ASI (Automatic Semicolon Insertion) convierte algunas secuencias de tokens en sentencias completas automáticamente, lo que es una fuente conocida de bugs sutiles.
- **Literales soportados:** number (IEEE 754 doble precisión para todo). BigInt (sufijo `n`: `9007199254740993n`). string (comillas simples, dobles o template literals con backtick y `${expresión}`). boolean (true/false). null, undefined, Symbol, RegExp literal (`/patrón/flags`).
- **Delimitadores:** Llaves `{}` para bloques. El punto y coma `;` es opcional gracias a ASI, pero su omisión puede generar comportamiento inesperado. Arrow functions (`=>`) permiten bloques o expresiones directas. Los módulos ES2015 usan import/export. El motor V8 compila JavaScript a código nativo mediante JIT (Just-In-Time compilation) usando optimizaciones especulativas.

Gramática BNF simplificada — estructuras esenciales en JavaScript:

```
FunctionDecl ::= ("async"? "function" IDENT? "(" ParamList? ")" Block

ArrowFunc    ::= ("async"? "(" ParamList? ")" "=>" (Block | Expr)

IfStmt       ::= "if" "(" Expr ")" (Stmt | Block)
              ("else" (IfStmt | Stmt | Block))?

WhileStmt     ::= "while" "(" Expr ")" (Stmt | Block)

ForStmt       ::= "for" "(" ForInit ";" Expr? ";" Expr? ")" (Stmt | Block)

ForOfStmt     ::= "for" "(" ("const"|"let"|"var") IDENT "of" Expr ")" (Stmt | Block)

AwaitExpr     ::= "await" Expr // Solo en contexto async
```

Código fuente JavaScript — Conjetura de Collatz:

```
"use strict";

function collatzPasos(n) {

    let pasos = 0;

    while (n !== 1) {

        n = n % 2 === 0 ? n / 2 : 3 * n + 1;

        pasos++;

    }

    return pasos;

}

const N = 100_000, inicio = performance.now();

let maxP = 0, numMax = 1;

for (let i = 1; i < N; i++) {

    const p = collatzPasos(i);

    if (p > maxP) { maxP = p; numMax = i; }
```

```
}
```

```
console.log(`Tiempo: ${performance.now()-inicio}.toFixed(2)} ms`);
```

Tabla de Benchmarking — Conjetura de Collatz (n < 100 000)

Hardware: CPU Intel Celeron (2 núcleos, ~1.9 GHz), 4 GB RAM LDDR4, Python, JavaScript .

Zig y Rust: tiempos de ejecución reales medidos.

Lenguaje	Paradigma Dominante	Mecanismo de Ejecución	Tiempo Ejec. (ms)	Mem. Pico (MB)
Zig 0.13.0	Imperativo / Estructurado	Compilación Nativa (LLVM) -O ReleaseFast	130.88 ms	2,5 *
Python 3.12	Multiparadigma (OO, Imperativo)	Interpretado (CPython / VM)	2173.25 ms	8,6
Rust 1.96.0	Multiparadigma (Funcional, Imperativo)	Compilación Nativa (LLVM) --release	103.20 ms	3,2 *
JavaScript (Node 20)	Multiparadigma (Prototípico, Funcional)	JIT (Just-In-Time) / V8 Engine	463.16 ms	22,1

Las pruebas validan la jerarquía de rendimiento: Nativo > JIT > Interpretado, incluso en hardware de bajos recursos (Celeron), con las siguientes claves:

- Rust (1º) vs. Zig (2º): Rust superó a Zig contra todo pronóstico debido a que las optimizaciones de su compilador (LLVM) son más agresivas para esta arquitectura específica, demostrando que el rendimiento depende del contexto y no es absoluto.
- JavaScript (3º): Obtuvo un resultado sólido para ser un lenguaje dinámico. Aunque su motor JIT (V8) es eficiente, el hardware limitado hace que el proceso de optimización inicial (warm-up) sea más lento.
- Python (4º): Fue el más lento debido a las limitaciones estructurales de su intérprete (CPython) y el GIL, que le impide aprovechar los núcleos del procesador.

Conclusión: En hardware modesto, las distancias de tiempo entre lenguajes se acortan, pero se mantiene la ventaja de los lenguajes compilados sobre los interpretados.

Diseño de Lenguaje L (DSL)

Componentes del Alfabeto y Literales (Morfología Básica)

Categoría	Descripción / Regla	Ejemplos válidos
Identificadores	Nombres de variables o componentes. Deben comenzar con una letra minúscula y pueden contener letras, números y guiones bajos. No puede haber separaciones con espacio en el nombre de la variable.	bateria_1, sensorFlujo, refrigeracion, x, y.
Literales Numéricos	Números enteros o decimales. Los decimales se escriben con un punto “ . ”.	55, 20.5, 90, -12
Literales Booleanos	Valores lógicos de dos estados para interruptores y relés. Se utiliza correct para el estado afirmativo/alto y incorrect para el estado negativo/bajo.	correct, incorrect
Operadores Aritméticos	Símbolos matemáticos básicos para realizar cálculos de balance de energía si fuera necesario.	+, -, *, ^, /, //, ++, --,
Operadores Relacionales	Símbolos para comparar valores en los condicionales.	>, <, ==, >=, <=, !=, ===, !==
Operadores Lógicos	Palabras o símbolos para conectar múltiples condiciones en una sola sentencia. inv se usa para invertir un estado lógico.	and, or, inv
Delimitadores	Símbolos para agrupar argumentos o finalizar sentencias. Punto y coma: Fin de sentencia.	() { } , ;

	Paréntesis: Paso de argumentos o parámetros. Llaves: Apertura { y cierre } de bloques de instrucciones Coma: Separador de elementos.	
Comentarios	Líneas de texto que el analizador léxico ignorará por completo, usadas para documentar el código.	# Esto es un comentario

Palabras Clave y Primitivas Obligatorias (Diccionario del DSL)

Token (Palabra Reservada)	Tipo de Token	Función / Significado Técnico
init_grid	Primitiva de Inicialización	Arranca y conecta el sistema de la microred inteligente.
leer_temperatura	Primitiva de Lectura (Sensor)	Obtiene en tiempo real los grados Celsius de un contenedor de baterías específico.
estado_carga	Primitiva de Lectura (Sensor)	Devuelve el porcentaje de energía disponible (0% a 100%) en un banco de litio.
leer_potencia	Primitiva de Lectura (Sensor)	Mide el flujo eléctrico de entrada o salida en kilovatios (kW) de inversores o paneles.
conmutar_linea	Primitiva de Acción (Actuador)	Envía una señal al driver de bajo nivel para conectar o aislar los relés de un sector.
if	Estructura de Control	Inicia un bloque de evaluación condicional.
or_if	Estructura de Control	Permite evaluar una condición secundaria si la anterior falló.

<code>otherwise</code>	Estructura de Control	Define el bloque de instrucciones por defecto que se ejecutará si ninguna de las condiciones anteriores se cumplió.
<code>while</code>	Estructura de Control	Inicia un bucle repetitivo basado en una condición lógica.
<code>{</code>	Delimitador de Apertura de Bloque	Delimita el inicio de las instrucciones a ejecutar.
<code>}</code>	Delimitador de Cierre de Bloque	Marca el cierre definitivo de un bloque lógico o de una estructura de control (<code>if</code> , <code>or_if</code> , <code>otherwise</code> o <code>while</code>).
<code>print</code>	Primitiva de Sistema	Muestra un mensaje o alerta en la interfaz Hombre-Máquina (HMI) del operador.

Gramática Sintáctica Abstracta

1. Reglas de Estructura Global del Programa.

- a. **Regla de Inicialización:** Todo script debe comenzar estrictamente con la primitiva del sistema “init_grid”, seguida del delimitador de fin de sentencia `;`. Ninguna otra instrucción puede ejecutarse antes de esta sincronización de hardware.
- b. **Regla de Secuenciación:** Las sentencias se ejecutan de forma secuencial (de arriba hacia abajo). Cada instrucción operativa, asignación o lectura de sensor debe finalizar obligatoriamente con el delimitador `;`.

2. Reglas de Asignación y Expresiones Matemáticas.

- a. **Estructura de Asignación:** Un bloque de asignación válido debe seguir estrictamente el orden: `[Identificador de Variable] = [Expresión o Llamada a Sensor];`.
- b. **Precedencia de Operadores Aritméticos:** Al resolver fórmulas complejas (como balances de potencia), el compilador evaluará los operadores con el siguiente orden de prioridad estricto:
 - i. Operador de potenciación (^).
 - ii. Multiplicación (*) y División (/).
 - iii. Suma (+) y Resta (-). *Los paréntesis () se utilizan para romper esta precedencia y forzar la evaluación prioritaria de una subexpresión.*
- c. **Regla de Operadores de Modificación Unaria (++ / --):** Los operadores de incremento (++) y decremento (--) actúan como expresiones unarias de asignación compacta.
 - i. Su sintaxis válida obliga a colocarlos inmediatamente después de un identificador de variable numérica (ej. `ciclos++`; o `intentos--`;
 - ii. A nivel sintáctico, poseen la máxima prioridad de evaluación matemática, ejecutándose de forma inmediata antes de procesar cualquier otra operación relacional o aritmética adyacente dentro de la misma sentencia.

3. Reglas de Flujo de Control (Condicionales y Bucles).

- a. **Estructura Condicional Completa (if / or_if / otherwise):**
 - i. Debe iniciar con el token `if`, seguido de una condición lógica.
 - ii. El cuerpo de instrucciones debe abrirse con `{` y cerrarse con `}`.

- iii. En caso de evaluar rutas alternativas, se puede encadenar el token `or_if` (con su propia condición y su propio bloque entre llaves).
- iv. Si existen acciones por defecto, se cierra con el token `otherwise` seguido de un último bloque entre llaves `{ }`.

b. Estructura de Monitoreo Cíclico (`while`):

- i. Debe iniciar con el token `while`, seguido de una condición lógica (o el literal de bucle infinito `correct`).
- ii. El bloque de instrucciones repetitivas debe estar estrictamente encapsulado entre una llave de apertura `{` y una llave de cierre `}`.

4. Reglas de Inversión Lógica

- a. **Regla del Operador Unario:** El operador de inversión lógica (`inv`) posee la máxima prioridad sintáctica en las expresiones booleanas. Debe colocarse inmediatamente antes de un identificador de estado o de una llamada a un sensor booleano.
- b. **Evaluación:** Al ser procesado, transforma sintácticamente un valor `correct` en `incorrect`, y viceversa.

Codigo del Escenario A (Prevención de Fuga Termica)

Se inicia el sistema ECO-GRID

init_grid;

Ciclo infinito para monitorear en tiempo real ambas baterías


```

while correct {

    // Se obtiene las lecturas térmicas de los sensores de las baterías

    tempBatPrincipal = leer_temperatura(bat_01);

    tempBatRespaldo = leer_temperatura(bat_02);


    # Se evalúa el umbral crítico de seguridad

    if tempBatPrincipal > 55 {

        conmutar_linea(refrigeracion_aux, correct);    # Activa sistemas de refrigeración auxiliar
        conmutar_linea(carga_solar, incorrect);        # Detiene el ingreso de energía
        conmutar_linea(sector_industrial, incorrect);   # Aislamiento físico del sector industrial
        conmutar_linea(red_comercial, correct);         # Desvía el consumo al respaldo externo


        print("ALERTA: Fuga térmica detectada. Protocolo de refrigeración activado. Desvío del
        consumo al respaldo externo.");

    }

    or_if temp_respaldo > 45 {

        conmutar_linea(refrigeracion_aux2, correct);

        print("PRECAUCIÓN: Batería de respaldo tiene temperatura muy alta. Sistema
        preventivo de enfriamiento activado.")

    }

    otherwise {

        if inv temp_principal > 55 {

            conmutar_linea(refrigeracion_aux, incorrect);

            # Desactiva extractores de emergencia

        }

    }

}

```

```
}
```

Código de escenario B (Balance de Carga y Venta de Energía)

```
# Se inicializa el sistema ECO-GRID
```

```
init_grid;
```

```
# Se obtienen los datos actuales de los sensores
```

```
carga_actual = estado_carga(bat_01);
```

```
generacion_solar = leer_potencia(paneles_sector_A);
```

```
demanda_interna = leer_potencia(inversor_principal);
```

```
# Caso 1: Excedente de energía (Batería óptima > 90% y producción > demanda)
```

```
if carga_actual > 90 and generacion_solar > demanda_interna {
```

```
    conmutar_linea(reles_automaticos, correct); # Acciona relés para vender energía
```

```
    print("Sistema óptimo: Inyectando excedente a la red pública general.");
```

```
}
```

```
# Caso 2: Baterías debajo del límite crítico (< 20% de carga) durante alta demanda nocturna
```

```
or_if carga_actual < 20 {
```

```
    conmutar_linea(sectores_no_esenciales, incorrect); # Aísla sectores no esenciales
```

```
    conmutar_linea(sector_medico, correct);           # Preserva áreas críticas
```

```
    conmutar_linea(sector_servidores, correct);       # Preserva infraestructura clave
```

```
    print("AVISO: Carga crítica. Sectores no esenciales aislados.");
```

```
}
```

```
# Caso 3: Estado normal
```

```
otherwise {
```

```
    print("Sistema operando en rangos estables.");
```

```
}
```

Conclusión

El recorrido analítico efectuado a lo largo de este informe pone de manifiesto que la evolución de los lenguajes de programación no es un proceso lineal, sino una ramificación de soluciones de compromiso que balancean el control directo sobre el hardware, la expresividad semántica y la corrección matemática. Ningún paradigma posee la supremacía absoluta; por el contrario, cada uno redefine la frontera de lo computable y lo gestionable bajo una filosofía propia. El paradigma imperativo destaca por su eficiencia bruta al mapear instrucciones directamente sobre el procesador, pero hereda la fragilidad de la explosión de estados y los efectos secundarios destructivos. Frente a esto, la orientación a objetos aporta un marco robusto de abstracción y preservación de invariantes de estado, mientras que el paradigma funcional y el lógico elevan la programación hacia un plano formal donde el código se asimila a ecuaciones o relaciones puras, habilitando la transparencia referencial y la optimización avanzada por compilador.

Por otra parte, el abordaje de la concurrencia contemporánea confirma que los enfoques tradicionales basados en memoria compartida mutable resultan insostenibles y propensos a condiciones de carrera en sistemas de alta escala. En este escenario, el Modelo de Actores demuestra ser una solución de alta elegancia arquitectónica. Al sustituir los cerrojos y la sincronización manual por el aislamiento estricto y el paso de mensajes asíncronos, este modelo no solo erradica los problemas de concurrencia por diseño, sino que introduce propiedades indispensables en la infraestructura moderna, tales como la tolerancia reactiva a fallos mediante árboles de supervisión y la transparencia de ubicación en entornos distribuidos.

En última instancia, el análisis sintáctico y el *benchmarking* empírico de la Conjetura de Collatz materializan de forma contundente la distancia que media entre el diseño teórico de un lenguaje y su comportamiento real en tiempo de ejecución. Las profundas variaciones observadas entre lenguajes nativos de control de bajo nivel (como Zig y Rust), lenguajes interpretados (como Python) y entornos con optimización dinámica JIT (como JavaScript bajo

V8) evidencian que el rendimiento es multifactorial. Factores como el *layout* de memoria (pila frente a montículo) , la rigidez gramatical y los costes de abstracción determinan la viabilidad técnica de un sistema. En consecuencia, para el futuro profesional de la Ingeniería en Informática de la UNEG, el dominio profundo de estos paradigmas y de sus mecanismos de compilación e inferencia deja de ser un ejercicio puramente teórico para convertirse en una competencia crítica indispensable para el diseño de software eficiente, seguro y escalable.

Referencias bibliográficas

Blandy, J., Orendorff, J., & Tindall, L. F. S. (2021). Programming Rust: Fast, safe systems development (2.^a ed.). O'Reilly Media.

<https://www.oreilly.com/library/view/programming-rust-2nd/9781492052586/>

Zig Software Foundation. (2024). Zig language reference (versión 0.13).

<https://ziglang.org/documentation/0.13.0/>

Python Software Foundation. (2024). The Python language reference (versión 3.12).

<https://docs.python.org/3.12/reference/>

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). Compilers: Principles, techniques, and tools (2.^a ed.). Pearson Addison-Wesley. <https://dl.acm.org/doi/book/10.5555/1177220>

Wikipedia. (3 de junio de 2026). Domain-specific language. En Wikipedia.

https://en.wikipedia.org/wiki/Domain-specific_language

EECS 390. (s. f.). Foundations. En Programming Language Principles and Paradigms.

<https://eecs390.github.io/notes/foundations.html>

Cueva Lovelle, J. M. (2001). Lenguajes, gramáticas y autómatas (2.^a ed.). Universidad de Oviedo. <https://reflection.uniovi.es/ortin/publications/automata.pdf>